

Guía de Estudio — Tema 7

Máquinas de Vectores de Soporte (Support Vector Machines, SVM)

Maestría en Inteligencia Artificial — UNIR · Técnicas de Aprendizaje Automático · Mayo 2026

Contenido

1. Introducción y contexto histórico
2. Conceptos geométricos fundamentales
3. Clasificador de margen máximo (Hard Margin)
4. Clasificador de margen suave (Soft Margin) y el parámetro C
5. El truco del kernel
6. Funciones de kernel: lineal, polinomial, RBF, sigmoide
7. Multiplicadores de Lagrange y formulación dual (intuición)
8. Aspectos prácticos: escalamiento, hiperparámetros y SVR
9. Tabla comparativa SVM vs otros modelos
10. Banco de preguntas tipo examen con justificación

1. Introducción y contexto histórico

Las **Máquinas de Vectores de Soporte (SVM — Support Vector Machines)** son un algoritmo de aprendizaje supervisado propuesto inicialmente por Vladimir Vapnik en 1963 y formalizado por Cortes y Vapnik en 1995. Resuelven originalmente problemas de **clasificación binaria** y posteriormente fueron extendidas a regresión bajo el nombre **SVR (Support Vector Regression)**.

Cambio de paradigma histórico: La gran contribución de Vapnik fue proponer un modelo de predicción *no basado en supuestos probabilísticos*. Hasta ese momento, prácticamente todos los modelos estadísticos (regresión lineal, logística, LDA, Naive Bayes) asumían una distribución de los datos. SVM rompió con ese paradigma: el problema se modela **geométricamente** y se resuelve mediante **optimización convexa con restricciones**.

1.1. ¿Por qué SVM y no solo modelos lineales?

Los modelos lineales (regresión lineal del Tema 4, regresión logística) son simples, interpretables y eficientes. Pero su poder de predicción es limitado: solo capturan relaciones lineales entre features y target.

Los modelos no lineales tradicionales presentan varios problemas:

- **Interpretabilidad reducida** por la complejidad inherente.
- **Sobreajuste (overfitting)** más frecuente — conectado al bias-variance tradeoff del Tema 4.
- **Requieren más datos** para generalizar.
- **Coste computacional alto**, especialmente en alta dimensionalidad.

SVM resuelve este dilema: **combina la simplicidad de un clasificador lineal en un espacio transformado con la capacidad de modelar fronteras no lineales en el espacio original**, sin pagar el costo computacional de operar explícitamente en alta dimensión. Esto es posible gracias al *kernel trick*, que veremos en detalle.

2. Conceptos geométricos fundamentales

2.1. Hiperplano

Definición de hiperplano:

Un **hiperplano** en un espacio de dimensión n es un subespacio de dimensión

$n - 1$ que divide el espacio en dos mitades.

Dimensión del espacio	Hiperplano
1D (una variable)	Un punto
2D (dos variables)	Una recta
3D (tres variables)	Un plano ordinario
nD ($n \geq 4$ variables)	Un hiperplano de dimensión $n - 1$

Ecuación general del hiperplano en un espacio de n dimensiones:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n = 0$$

donde el vector $\beta = (\beta_1, \beta_2, \dots, \beta_n)$ se denomina **vector normal**: apunta en

dirección ortogonal a la superficie del hiperplano. Para que las distancias sean interpretables, se

exige que β sea unitario:

$$\sum_{j=1}^n \beta_j^2 = 1$$

Intuición de la proyección:

Si se proyecta cualquier punto x sobre el vector normal β , el valor

obtenido es **proporcional a la distancia perpendicular del punto al hiperplano**:

- Puntos sobre el hiperplano $\rightarrow$ proyección = 0
- Puntos por encima del hiperplano $\rightarrow$ proyección > 0
- Puntos por debajo del hiperplano $\rightarrow$ proyección < 0
- Cuanto más lejos del hiperplano, mayor es el valor absoluto de la proyección.

Esta característica geométrica es el corazón de SVM.

2.2. Margen (Margin)

Definición de margen:

El **margen** es la distancia perpendicular entre el hiperplano de separación y los puntos de datos más cercanos de cada clase. SVM se denomina a veces **clasificador de margen máximo** porque busca el hiperplano que *maximiza* esta distancia.

La intuición es directa: un margen amplio actúa como una "zona segura" que aleja la frontera de decisión de los datos de entrenamiento, lo que **mejora la generalización** y reduce el riesgo de sobreajuste.

2.3. Vectores de soporte (Support Vectors)

Definición de vector de soporte:

Los **vectores de soporte** son los puntos de datos del conjunto de entrenamiento que se encuentran *exactamente* sobre el margen (o, en soft margin, dentro o cruzando el margen). Estos puntos son los únicos que determinan la posición y orientación del hiperplano.

Una propiedad fascinante de SVM: si elimináramos cualquier punto que *no* sea support vector, el hiperplano resultante sería idéntico. Solo los support vectors "soportan" la frontera de decisión. De ahí el nombre del algoritmo.

3. Clasificador de margen máximo (Hard Margin)

3.1. Etiquetado de clases en SVM

A diferencia de otros clasificadores binarios que usan $y \in \{0, 1\}$, SVM utiliza la convención:

$$y_i \in \{-1, +1\}$$

Esta elección no es arbitraria. Permite escribir la condición de "estar bien clasificado" de forma elegante:

$$y_i \cdot (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) > 0$$

Si $y_i = +1$ y el punto cae del lado positivo del hiperplano, el producto es positivo. Si

$y_i = -1$ y el punto cae del lado negativo, el producto también es positivo. El **signo**

del producto indica si el punto está bien clasificado, independientemente de la clase.

3.2. Formulación matemática del Hard Margin

Problema de optimización del clasificador de margen máximo:

$$\max_{\beta_0, \beta_1, \dots, \beta_p} M$$

sujeto a:

$$\sum_{j=1}^p \beta_j^2 = 1$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M \quad \text{para todo } i = 1, \dots, N$$

Lectura por piezas:

- M : el margen — la cantidad a maximizar. Es la distancia entre el hiperplano y los support vectors.

- $\sum_{j=1}^p \beta_j^2 = 1$: normalización del vector normal. Garantiza solución única y

vuelve a M una distancia euclidiana interpretable.

- $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M$: condición de margen — cada punto debe estar al menos a distancia M del hiperplano, del lado correcto.

Limitación crítica del Hard Margin:

El clasificador de margen máximo **solo funciona si los datos son perfectamente separables linealmente**. Si hay un solo punto que no puede ser separado, el problema de optimización *no tiene solución*. Esto lo vuelve poco práctico en datasets reales con ruido. La solución es el **soft margin**.

4. Clasificador de margen suave (Soft Margin) y el parámetro C

4.1. La idea: permitir violaciones controladas

En la práctica, casi ningún dataset real es perfectamente separable. La solución es introducir **variables de holgura** (slack variables) ϵ_i , una por cada punto, que miden cuánto se viola la restricción de margen para ese punto.

4.2. Formulación matemática del Soft Margin

Problema de optimización con margen suave (convención UNIR/ISLR):

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

sujeto a:

$$\sum_{j=1}^p \beta_j^2 = 1$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i)$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C$$

4.3. Interpretación geométrica de las slack variables ϵ_i

Valor de ϵ_i	Interpretación geométrica
$\epsilon_i = 0$	El punto i está del lado correcto del margen (bien clasificado y fuera de la zona de tolerancia).

$0 < \epsilon_i \leq 1$	El punto está dentro del margen pero del lado correcto del hiperplano (bien clasificado pero "intruso" en la zona de seguridad).
$\epsilon_i > 1$	El punto está del lado equivocado del hiperplano (mal clasificado).

4.4. El parámetro C — punto crítico de confusión

⚠ **Atención:** el parámetro C tiene DOS convenciones distintas.

Esto es la fuente más común de errores en exámenes. Hay que saber distinguir cuál se está usando.

Convención 1: UNIR / ISLR (la del PDF del tema)

C aparece como **límite superior** del presupuesto total de violaciones:

$$\sum_{i=1}^n \epsilon_i \leq C$$

Valor de C	Efecto
C grande	Mucho presupuesto para violar el margen → margen más amplio → más tolerancia a errores → posible underfitting
C pequeño	Poco presupuesto → margen más estrecho → modelo casi como hard margin → posible overfitting

Convención 2: scikit-learn (la implementación en Python)

C aparece como **peso de la penalización** en la función objetivo:

$$\min_{w, b, \epsilon} \frac{1}{2} \|w\|^2 + C \cdot \sum_{i=1}^n \epsilon_i$$

Valor de C	Efecto
C grande	Penalización fuerte de cada violación → modelo intolerante a errores → margen más estrecho → overfitting
C pequeño	Penalización débil → tolera errores → margen amplio → underfitting

Regla mnemotécnica definitiva:

- Si C es **límite superior**
de los errores (convención UNIR/ISLR), C grande *permite* más errores → margen amplio.
- Si C es **peso del castigo**
(convención sklearn), C grande *castiga* más los errores → margen estrecho.

En las preguntas del test de la asignatura, interpretar siempre desde la **convención UNIR/ISLR**.

Ejemplo conceptual de la balanza (sklearn):

Pensar la función objetivo como una balanza con dos platos:

- Plato 1:

$$\frac{1}{2} \|w\|^2 \quad \text{— maximizar el margen (el margen es } 2/\|w\| \text{ , por lo}$$

que minimizar $\|w\|^2$ equivale a maximizar el margen).

- Plato 2:

$$\sum \epsilon_i \quad \text{— minimizar las violaciones.}$$

C es el peso que se pone en el plato 2. Cuanto más grande, más fuerza el modelo a hacer

$$\sum \epsilon_i \quad \text{pequeño, sacrificando margen.}$$

5. El truco del kernel (Kernel Trick)

5.1. El problema: fronteras no lineales

Incluso con soft margin, hay datasets donde **ninguna recta puede separar las clases**, no importa cuánto se ajuste C. Por ejemplo, datos dispuestos en círculos concéntricos.

Una solución intuitiva: **transformar los datos a un espacio de mayor dimensión** donde sí sean

linealmente separables, y aplicar SVM allí. Por ejemplo, si $x = (x_1, x_2)$ está en 2D,

podemos transformar a 6D:

$$\phi(x) = (x_1, x_2, x_1^2, x_2^2, x_1 x_2, \sqrt{2} x_1 x_2)$$

El problema: **esta transformación explícita es computacionalmente costosa**, especialmente en alta dimensión. Con polinomios de grado alto o muchas features originales, el espacio transformado puede tener millones de dimensiones.

5.2. La idea brillante de Vapnik

Al derivar matemáticamente SVM en su **forma dual** (usando multiplicadores de Lagrange), aparece algo extraordinario:

Propiedad fundamental:

La función de decisión de SVM y todo el problema de optimización **solo requieren productos punto entre pares de vectores**, nunca los vectores por separado:

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i y_i \langle x, x_i \rangle$$

donde S es el conjunto de support vectors, α_i son los multiplicadores de

Lagrange, y $\langle x, x_i \rangle$ es el producto punto.

Si lo único que necesitamos es el producto punto, entonces surge la pregunta:

¿Y si pudiéramos calcular el producto punto en el espacio transformado, sin transformar los datos?

5.3. Definición formal del kernel

Función kernel:

Una función $K(x_i, x_j)$ es un **kernel** si existe una función de transformación

ϕ tal que:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

Es decir, el kernel devuelve un número que es **equivalente al producto punto en el espacio transformado**, pero calculado *sin transformar nada*.

5.4. Ejemplo numérico que destrava la intuición

Ejemplo: kernel polinomial de grado 2 en 2D

Sean $x = (x_1, x_2)$ y $z = (z_1, z_2)$ dos puntos en 2D. Definimos la transformación a 3D:

$$\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Producto punto en 3D (forma cara):

$$\langle \phi(x), \phi(z) \rangle = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2$$

Esta expresión factoriza:

$$x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 = (x_1 z_1 + x_2 z_2)^2$$

Y $x_1 z_1 + x_2 z_2 = \langle x, z \rangle$. Entonces:

$$K(x, z) = \langle x, z \rangle^2 = \langle \phi(x), \phi(z) \rangle$$

Conclusión:

calculamos el producto punto en 3D *sin tocar el espacio 3D*. Solo hicimos: (1) producto punto en 2D, y (2) elevar al cuadrado.

5.5. Por qué esto es revolucionario

Con 1000 features y un kernel polinomial de grado 10, la transformación explícita generaría un espacio de millones de dimensiones — inviable. Con el kernel trick, basta con: (1) un producto punto en 1000D, y (2) elevar al exponente 10. **Microsegundos.**

Aclaración conceptual crítica:

El kernel **no transforma los datos**. Los datos siguen viviendo en el espacio original. Solo las *similitudes calculadas entre pares de puntos* se comportan como si los datos estuvieran en un espacio expandido. Por eso se llama *kernel*

trick

: es un truco matemático, no una transformación real.

6. Funciones de kernel

6.1. Resumen de los kernels más usados

Kernel	Fórmula	Hiperparámetros
Lineal	$K(x, x_i) = x \cdot x_i$	Ninguno
Polinomial	$K(x, x_i) = (\gamma(x \cdot x_i) + c)^d$	γ, c, d
RBF (Gaussiano)	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$	γ
Sigmoide	$K(x, x_i) = \tanh(\gamma(x \cdot x_i) + c)$	γ, c

6.2. Kernel Lineal

Es simplemente el producto punto en el espacio original. Equivale a no usar kernel — útil cuando los datos ya son aproximadamente separables linealmente, y muy eficaz en problemas de alta dimensionalidad como clasificación de texto.

6.3. Kernel Polinomial

Transforma los datos a un espacio de polinomios de grado d . Captura interacciones polinomiales entre features. Útil cuando se sospecha que la relación es de tipo polinomial.

6.4. Kernel RBF (Radial Basis Function, también llamado Gaussiano)

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Propiedad asombrosa del RBF:

El kernel RBF corresponde a una transformación implícita ϕ a un **espacio de dimensión infinita**. Si quisiéramos transformar explícitamente los datos al espacio donde el RBF calcula el producto punto, necesitaríamos vectores con infinitas componentes.

Imposible. Pero el kernel trick lo resuelve en $O(d)$ operaciones por par.

Interpretación del parámetro γ (gamma)

γ controla el **ancho de la "campana gaussiana"** centrada en cada support vector.

Pensemos cada support vector como una fuente de luz:

	Comportamiento	Riesgo
--	----------------	--------

Valor de γ		
γ pequeño	La luz se esparce lejos. Cada support vector "ilumina" región amplia. Fronteras suaves, casi lineales.	Underfitting (alto bias)
γ grande	La luz se concentra cerca. Cada support vector solo "ilumina" su vecindad. Fronteras complejas, llenas de islas locales.	Overfitting (alta varianza)

Caso límite con $\gamma \rightarrow 0$:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \approx \exp(0) = 1$$

Para casi todos los pares, $K \approx 1$. El modelo "ve" todos los puntos como casi

iguales: pierde capacidad de distinguir estructuras locales, la frontera se vuelve extremadamente suave, casi lineal $\rightarrow$ alto bias.

6.5. Kernel Sigmoide

$$K(x, x_i) = \tanh(\gamma(x \cdot x_i) + c)$$

Tiene forma similar a una capa de red neuronal. Produce valores en $(-1, 1)$. En la práctica se usa raramente — suele ser menos efectivo y más propenso a overfitting que el RBF.

7. Multiplicadores de Lagrange y formulación dual (intuición)

7.1. Conexión con cálculo multivariable

Los **multiplicadores de Lagrange** son una técnica clásica para encontrar extremos (máximos o mínimos) de una función sujeta a restricciones. La fórmula básica:

$$\nabla f(x) = \lambda \nabla g(x)$$

donde f es la función a optimizar, g es la restricción y λ es el multiplicador de Lagrange.

7.2. ¿Por qué aparecen en SVM?

SVM es exactamente un problema de optimización con restricciones:

- **Función a optimizar:** minimizar $\frac{1}{2} \|w\|^2$ (equivalente a maximizar el margen).
- **Restricciones:** $y_i(w \cdot x_i + b) \geq 1 - \epsilon_i$ para cada punto i , más $\epsilon_i \geq 0$.

Como hay n restricciones (una por cada punto del dataset), se necesitan n multiplicadores de Lagrange, denotados α_i .

7.3. La ecuación que conecta todo

Al resolver SVM mediante lagrangianos, aparece esta relación fundamental:

$$w = \sum_{i=1}^n \alpha_i y_i x_i$$

El vector normal w del hiperplano es una **combinación lineal de los puntos de entrenamiento**, ponderados por los multiplicadores α_i .

Propiedad clave:

La mayoría de los α_i resultan ser exactamente **cero**. Solo los puntos con

$\alpha_i > 0$ sobreviven en la suma. **Esos puntos con $\alpha_i > 0$ son**

precisamente los support vectors. Todos los demás puntos del dataset son completamente irrelevantes para definir el hiperplano.

Insight de cierre:

SVM se llama "Support Vector Machine" porque los multiplicadores de Lagrange *iluminan* qué puntos verdaderamente *soportan* la frontera de decisión. Si se elimina cualquier punto que no sea support vector, el hiperplano resultante es idéntico.

8. Aspectos prácticos

8.1. Escalamiento: crítico para SVM

SVM es altamente sensible al escalamiento de las variables.

Esto es especialmente cierto para el kernel RBF, que depende directamente de la distancia

euclidiana $\|x_i - x_j\|^2$. Sin escalamiento, una variable con rango grande

domina la distancia y aplasta la influencia de las demás.

Ejemplo clásico del problema sin escalar:

- Variable 1: edad, rango 0–100
- Variable 2: salario, rango 0–10,000,000

La distancia euclidiana $\|x_i - x_j\|^2$ estará dominada absolutamente por el

salario. La edad prácticamente desaparece del kernel. El modelo será incapaz de usar la información de la edad.

Solución: aplicar `StandardScaler` o `MinMaxScaler` antes de entrenar SVM.

Comparación con árboles de decisión (Tema 6):

Modelo	¿Sensible a escalamiento?	Razón
SVM, KNN, K-Means, PCA	Sí, críticamente	Dependen de distancias geométricas, productos punto, normas
Árboles de decisión, Random Forest, Gradient Boosting	No	Realizan cortes ortogonales sobre una variable a la vez; no comparan magnitudes entre variables

8.2. Hiperparámetros principales y su efecto

Hiperparámetro	Aplica a	Efecto
C (convención sklearn)	Todos los kernels	C grande $\rightarrow$ margen estrecho, overfitting. C pequeño $\rightarrow$ margen amplio, underfitting
γ	RBF, Polinomial, Sigmoide	γ grande $\rightarrow$ frontera compleja, overfitting. γ pequeño $\rightarrow$ frontera suave, underfitting
d (degree)	Polinomial	Grado del polinomio. d alto $\rightarrow$ fronteras más complejas
kernel	—	'linear', 'poly', 'rbf', 'sigmoid'. RBF es el default razonable

8.3. Support Vector Regression (SVR)

SVM se extiende a problemas de regresión bajo el nombre **SVR**. La idea geométrica:

- En SVM clasificación: encontrar el hiperplano que *maximiza el margen* entre clases.
- En SVR regresión: encontrar la función que pase lo más cerca posible de los puntos,

manteniendo el error dentro de un margen de tolerancia ϵ .

Los puntos que caen dentro del margen de tolerancia son los **vectores de soporte** de SVR. Las observaciones fuera del margen contribuyen al error y son penalizadas.

9. Tabla comparativa

9.1. Hard Margin vs Soft Margin

Concepto	Hard Margin	Soft Margin
¿Permite errores?	No	Sí, controlados por ϵ_i

Restricción	$y_i(w \cdot x_i + b) \geq 1$	$y_i(w \cdot x_i + b) \geq 1 - \epsilon_i$
¿Cuándo funciona?	Solo si los datos son linealmente separables	Siempre
Parámetro C	No existe	Controla el tradeoff margen-errores
Aplicación práctica	Prácticamente no se usa	El estándar en la industria

9.2. SVM Lineal vs SVM con Kernel

Concepto	SVM Lineal	SVM con Kernel
Frontera de decisión	Hiperplano recto	No lineal en el espacio original
Espacio donde separa	Original	Implícito de alta dimensión (vía kernel)
Complejidad	Baja	Mayor; controlable con C y γ
Cuándo usar	Datos casi linealmente separables; alta dimensionalidad (ej. texto)	Datos con estructura no lineal evidente

9.3. SVM vs Árboles de Decisión

Aspecto	SVM	Árboles de Decisión
Naturaleza	Modelo geométrico (distancias, hiperplanos)	Modelo basado en particiones (reglas)
Frontera	Suave (lineal o no lineal según kernel)	Cortes ortogonales — hiperrectángulos
Escalamiento	Crítico	No requiere
Interpretabilidad	Baja (especialmente con kernels)	Alta (reglas if-then)

Outliers	Sensible (especialmente hard margin)	Robusto
Extrapolación	Limitada	No extrapola

10. Banco de preguntas tipo examen con justificación

Pregunta 1:

¿Cuál de las siguientes afirmaciones sobre las máquinas de vector de soporte es **falsa**?

1. Las SVM son eficaces para clasificación binaria y se pueden extender a regresión.
2. El objetivo principal de SVM es encontrar un hiperplano de separación óptimo que maximice el margen entre clases.
3. SVM puede manejar conjuntos de alta dimensionalidad y es robusta frente al sobreajuste con técnicas como soft margin y optimización convexa.
4. Las SVM siempre requieren que los datos sean linealmente separables en el espacio de características.

Respuesta correcta: D.

Justificación:

La falsedad de D es el corazón mismo del kernel trick y del soft margin. Soft margin

permite trabajar con datos no separables (admite violaciones $\epsilon_i > 0$), y el kernel

trick permite construir fronteras no lineales en el espacio original mapeando implícitamente a un espacio donde sí son linealmente separables. Las opciones A, B y C son todas verdaderas.

Pregunta 2:

¿Cuál de las siguientes afirmaciones sobre los hiperplanos es **cierta**?

1. Un hiperplano solo se define para dos dimensiones.
2. Límite de decisión entre clases que separa de manera óptima las instancias de datos en espacio de características multidimensional.
3. El hiperplano en SVM siempre pasa por el origen en el espacio de características.
4. Un hiperplano busca el sobreajuste de las instancias de datos.

Respuesta correcta: B.

Justificación:

A es falsa porque un hiperplano se define en cualquier dimensión n como un

subespacio de dimensión $n - 1$. C es falsa porque la presencia del término

β_0 (intercepto) permite hiperplanos que no pasan por el origen. D es absurda: el

hiperplano busca *separar*, no sobreajustar. B describe correctamente la función del hiperplano en SVM.

Pregunta 3:

¿Cuál de las siguientes afirmaciones sobre los vectores en SVM es **falsa**?

1. Los vectores de soporte son puntos de datos que se encuentran más cerca del hiperplano de separación.
2. La posición y orientación del hiperplano están determinadas por los vectores de soporte.
3. Los vectores de soporte son cruciales para definir el margen.
4. Los vectores de soporte se seleccionan aleatoriamente a partir de los datos de entrenamiento.

Respuesta correcta: D.

Justificación:

Los support vectors *no* se seleccionan aleatoriamente: emergen de la resolución del problema de optimización. Son precisamente aquellos puntos para los cuales el

multiplicador de Lagrange $\alpha_i > 0$. El resto de los puntos del dataset tienen

$\alpha_i = 0$ y no influyen en el hiperplano. A, B y C describen correctamente las

propiedades de los support vectors.

Pregunta 4:

¿Cuál de las siguientes afirmaciones es **cierta**?

1. Para una serie de datos de entrenamiento de un problema binario existen múltiples hiperplanos posibles.
2. Solo existe un hiperplano posible para cada conjunto de datos.
3. Los hiperplanos hay que crearlos únicamente para realizar predicciones.
4. El objetivo de SVM es encontrar el hiperplano óptimo que maximice el margen sin importar el error de clasificación.

Respuesta correcta: A.

Justificación:

Dado un conjunto de datos linealmente separables, existen *infinitos* hiperplanos que los separan. El trabajo de SVM es elegir *el óptimo*, aquel que maximiza el margen. B es falsa por lo anterior. C es absurda — los hiperplanos definen la frontera de decisión, no solo predicen. D es falsa: en soft margin, sí importa el error de clasificación (controlado por C).

Pregunta 5:

¿Cuál de las siguientes afirmaciones sobre un clasificador de margen máximo es **falsa**?

1. El clasificador de margen máximo tiene como objetivo encontrar un hiperplano que maximice el margen entre clases.
2. Selecciona el hiperplano que mejor separa los datos y maximiza la distancia a los puntos más cercanos de cada clase.
3. Mejora la generalización maximizando la separación entre clases, reduciendo el riesgo de sobreajuste.
4. Siempre requiere que las clases sean perfectamente separables mediante un hiperplano.

Respuesta correcta: D.

Justificación:

El clasificador de margen máximo *hard margin* sí requiere separabilidad perfecta, pero esa es justamente su **limitación crítica** que motivó la creación del *soft margin*. En SVM moderna (soft margin) las clases no necesitan ser perfectamente separables. La pregunta es ambigua porque "clasificador de margen máximo" históricamente se refiere al hard margin, pero la opción D es la *falsa* en el contexto de SVM como algoritmo general. A, B y C son verdaderas.

Pregunta 6:

Indica cuál de las siguientes afirmaciones sobre soft margin es **falsa**.

1. SVM de soft margin permite cierta clasificación errónea introduciendo variables débiles que relajan la rigurosidad del margen.
2. Se utiliza cuando los datos de entrenamiento no son perfectamente separables, permitiendo un límite más flexible con datos ruidosos.
3. El parámetro de regularización C controla el equilibrio entre maximizar el margen y minimizar el error de clasificación.
4. SVM de soft margin siempre funciona mejor que SVM de hard margin en conjuntos linealmente separables.

Respuesta correcta: D.

Justificación:

"Siempre funciona mejor" es una afirmación absoluta que no se sostiene. En datos perfectamente separables sin ruido, hard margin puede tener el mismo desempeño o incluso mejor (no introduce holgura innecesaria). El soft margin gana cuando hay ruido o solapamiento entre clases, pero no *siempre*. A, B y C son verdaderas.

Pregunta 7:

¿Cuál de las siguientes afirmaciones sobre el parámetro C en SVM es **falsa**?

1. Controla el equilibrio entre maximizar el margen y minimizar el error en los datos de entrenamiento.
2. Un valor más pequeño de C conduce a un margen más amplio, pero puede dar lugar a más errores permitidos.
3. Un valor mayor de C da como resultado un margen más estrecho pero reduce los errores tolerados, lo que podría provocar sobreajuste.
4. Aumentar el valor de C siempre mejora el rendimiento del modelo en datos no vistos.

Respuesta correcta: D.

Justificación:

Aumentar C indefinidamente *no* mejora el desempeño en test. De hecho, C demasiado grande lleva a overfitting (en convención sklearn) o a underfitting (en convención UNIR). El óptimo se encuentra por validación cruzada. Las opciones A, B y C describen correctamente el comportamiento del parámetro en la convención sklearn.

Nota: Las opciones B y C asumen la convención sklearn (C grande $\rightarrow$ margen estrecho). En la convención UNIR, la lógica se invierte. La opción D, sin embargo, es falsa en cualquiera de las dos convenciones.

Pregunta 8:

La expansión en forma de polinomios:

1. Permite capturar relaciones no lineales entre las características originales al mapear los datos a un espacio de mayor dimensión.
2. Solo se puede realizar utilizando un kernel lineal.
3. Siempre mejora el rendimiento del modelo en comparación con un enfoque lineal.
4. Transforma automáticamente el espacio de características original a un espacio de mayor dimensión sin introducir complejidad adicional.

Respuesta correcta: A.

Justificación:

A es precisamente la definición de expansión polinomial: mapear a un espacio de mayor dimensión para capturar no linealidad. B es absurda — kernel lineal es lo opuesto. C es falsa por overfitting. D es falsa: una expansión polinomial explícita sí introduce complejidad (es el kernel trick el que la *evita*).

Pregunta 9:

¿Cuál de las siguientes afirmaciones es una **ventaja** de los kernels?

1. Permiten utilizar un menor volumen de información.
2. El coste computacional es menor al utilizar kernels.
3. Los kernels permiten obtener fronteras de decisión no lineales.
4. Los kernels reducen la dimensionalidad de los datos al mapearlos a un espacio de características mayor.

Respuesta correcta: C.

Justificación:

C es la ventaja central de los kernels: permiten fronteras de decisión no lineales sin necesidad de transformar explícitamente los datos. A es ambigua/falsa. B es falsa porque el coste comparado con un SVM lineal es *mayor*, aunque mucho menor que la transformación explícita. D es absolutamente falsa — los kernels mapean a espacios de *mayor* dimensión, no reducen dimensionalidad.

Pregunta 10:

¿Cuál de las siguientes afirmaciones sobre la función sigmoide es **falsa**?

1.

Se define como $k(x, x') = \tanh(\alpha x^T x' + c)$ donde α y

c son parámetros ajustables.

2. Puede ser útil para modelar relaciones no lineales y capturar patrones complejos.
3. Transforma los datos a un espacio de mayor dimensión usando una función sigmoide para facilitar la separación lineal.
4. Siempre produce mejor rendimiento que otros kernels como el lineal o el polinomial.

Respuesta correcta: D.

Justificación:

La afirmación "siempre produce mejor rendimiento" es falsa para cualquier kernel. El sigmoide es, de hecho, *menos* usado en práctica porque suele ser propenso a overfitting y a veces no cumple con las condiciones de Mercer (no siempre define un producto interno válido en algún espacio). El RBF es típicamente la elección por defecto. A, B y C son verdaderas.

Cierre y comentarios de sesión

Dudas y temas que merecen profundización post-examen:

- **Derivación completa de la forma dual:** entender cómo se llega a

$$w = \sum \alpha_i y_i x_i \quad \text{a partir del lagrangiano. Requiere optimización}$$

convexa y KKT conditions.

- **Teorema de Mercer:** qué funciones pueden ser kernels válidos. Una función

K es un kernel válido si y solo si es simétrica y semidefinida positiva.

- **Reproducing Kernel Hilbert Spaces (RKHS):** el formalismo matemático que justifica que el RBF corresponde a un espacio de dimensión infinita.

-

SVR en profundidad: formulación con ϵ -insensitive loss.

- **Conexión con XAI (tesis):** SHAP y LIME aplicados a SVM. Particularmente interesante porque SVM con kernel RBF es uno de los modelos menos interpretables — los métodos post-hoc son la única vía.

Insights de la sesión que vale la pena anclar:

- El producto punto es el lenguaje matemático universal de "similitud" entre vectores. Toda la geometría de SVM (y de PCA, KNN, K-Means) emerge de él.
- El kernel trick es un truco computacional, no una transformación real. Los datos nunca se mueven.
- El parámetro C tiene dos convenciones opuestas. Saberlo evita errores en exámenes y al leer literatura.
- SVM es un modelo geométrico; árboles son modelos basados en reglas. Esta es la diferencia fundamental que explica por qué SVM necesita escalamiento y los árboles no.
- Solo los support vectors importan: el resto del dataset es prescindible. Por eso SVM es robusto a datasets grandes con muchos puntos "fáciles".

Última actualización: Mayo 2026 — Tema 7 (SVM) completado.